F - The Fair Nut and Strings

Process

Description of the Question

1. every string is build by 'a' or 'b',
2. every string's length is n,
3. there is k strings in total,
4. all the string s are in between s and t in dictionary order.
5. define c as the number of all possible prefix of all k strings
6. the aim is to find largest c.

Challenges and Overcoming

After getting the hint, I know I have to transform this string question to a Trie problem since there are only 2 direction of generate new valid string, i.e. adding 'a' or adding 'b'. Therefore, we can construct a Trie in the following way:

1. the root node represent the empty string,
2. for each char in the string, if it is 'a', we go to the left child node, if it is 'b', we go to the right child node.

After doing this, what the question provide is 2 paths in the Trie, and we need to search for k possible paths in between these 2 paths and in the same time maximum the number of nodes those paths can cover. This is due to each node in the Trie is represent a possible prefix.

In order to maximum the number of nodes we cover, we can use greedy method, i.e. in each level of the Trie, we try to cover as many nodes as possible. However, to cover the nodes we have 2 constrians:

1. since we only have k paths to cover nodes, and in each level each path can only cover at most 1 node, we can only cover k nodes in each level.
2. we can only cover the node in between s and t.

Now the question is how to caculate how many nodes in each level that are i between s and t. We can consider s and t as a binary number ('a' -> 0, 'b' -> 1). Suppose in level i - 1, the number of nodes between s and t is diff + 1, then in the next level i, these diff nodes in the pervious level will have their child nodes, leading to 2 * (diff + 1) nodes in this level. However, we don't know for sure that the child nodes of left-most and right-most node in previous level are in the range.

Suppose in i - 1 level, the binary value of t is T_i-1 and s is S_i-1. The prev_diff = T_i-1 - S_i-1. In this level, T_i = 2 * T_i-1 + T[i], S_i = 2 * S_i-1 + S[i]. The diff = 2 * T_i-1 + T[i] - (2 * S_i-1 + S[i]) = 2 * (T_i-1 - S_i-1) + T[i] - S[i]. Therefore the number of nodes in between s and t i level i is 2 * (T_i-1 - S_i-1) + T[i] - S[i] + 1.

Accoding to above 2 constrain, if the maximum number of nodes we cover in level i is max(k, 2 * (T_i-1 - S_i-1) + T[i] - S[i] + 1).